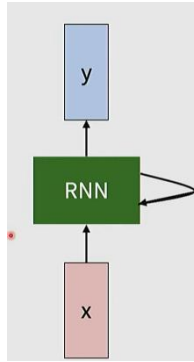


1. Recurrent Neural Network



Ý tưởng cốt lõi của RNN là có 1 chuỗi input x và có đầu ra là 1 chuỗi output y , mỗi input phụ thuộc vào quá khứ (gần giống HMM) -> RNN là mạng có hidden state giúp mô hình hoá phụ thuộc theo thời gian bằng cách truyền thông tin từ bước trước sang bước sau

2. RNN hidden state update

$$h_t = f_w(h_{t-1}, x_t)$$

$$y_t = f_{w_y}(h_t)$$

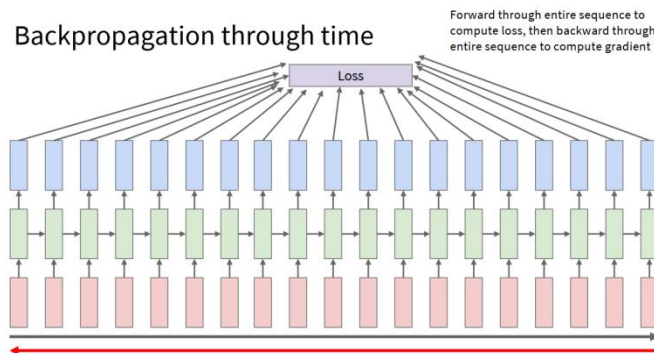
h_t là state mới, h_{t-1} là state cũ

Điểm cốt lõi: cùng một bộ tham số W dùng lại cho mọi timestep (weight sharing)

3. Backpropagation

3.1. Backpropagation Through Time

Ý tưởng: Forward qua toàn bộ chuỗi để tính loss từng cái xong rồi cộng tổng nó lại

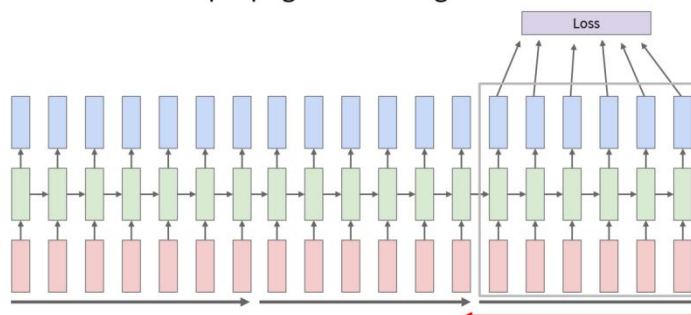


Điều này đồng nghĩa là sẽ có 1 bộ Weight sẽ được xài xuyên suốt không cập nhật vì khi chạy xong mới cập nhật -> có thể sẽ train nhiều epoch

3.2. Truncated Backpropagation Through Time

Ý tưởng: fix time window, tức là chọn số lượng các state rồi train và cập nhật weight xong tiếp tới time window tiếp theo -> cập nhật được liên tục hơn

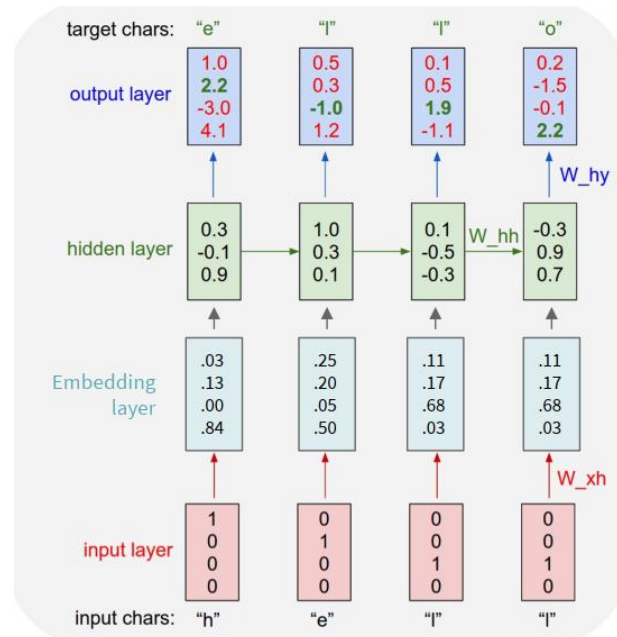
Truncated Backpropagation through time



4. Ví dụ thực tế: Character-level Language Model + Embedding

Vocabulary ví dụ: [h, e, l, o], training sequence “hello”

Sampling test-time: Mỗi bước softmax ra phân phối ký tự, sample 1 ký tự, feed lại làm input bước sau



Embedding layer = bảng tra (lookup table) biến mỗi token/ký tự (ID rời rạc) -> vector dense D chiều để NN/RNN xử lý được -> giảm chiều dữ liệu xuống D thay vì phải V(số vocab)

5. Searching for interpretable cells

Khi train RNN/LSTM trên chuỗi dài (nhất là code), một số cell sẽ học ra biến trạng thái dễ hiểu (in_string, in_comment, depth, line_pos, ...) -> interpretable cells vì activation của chúng tương ứng rõ ràng với một khái niệm

Ví dụ có tìm câu nói

"You mean to imply that I have nothing to eat out of... On the contrary, I can supply you with everything even if you want to give dinner parties," warmly replied Chichagov, who tried by every word he spoke to prove his own rectitude and therefore imagined Kutuzov to be animated by the same desire.

Kutuzov, shrugging his shoulders, replied with his subtle penetrating smile: "I meant merely to say what I said."

quote detection cell

6. RNN tradeoffs

- Ưu điểm

- + RNN xử lý được chuỗi có độ dài bất kỳ
- + RNN có thể dùng thông tin rất xa trong quá khứ
- + Tham số của RNN không phụ thuộc độ dài chuỗi
- + Weight sharing theo thời gian

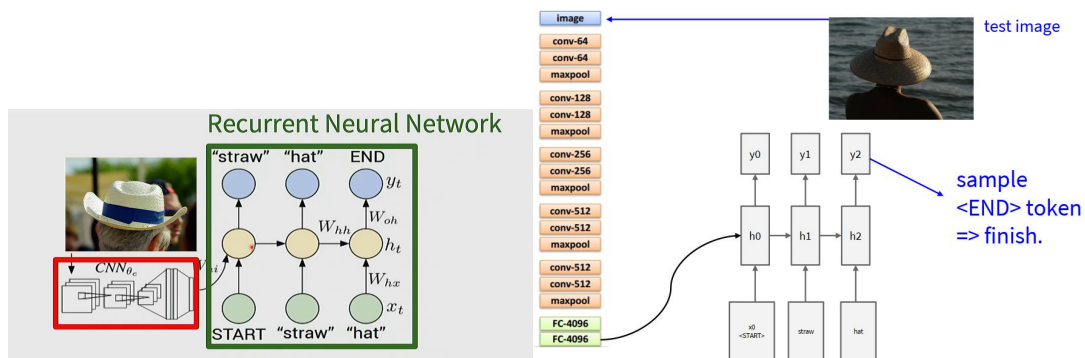
- Nhược điểm

- + RNN chậm vì phụ thuộc tuần tự giữa các timestep
- + Thực tế RNN khó học long-range dependency vì nếu đi xa thì các gradient

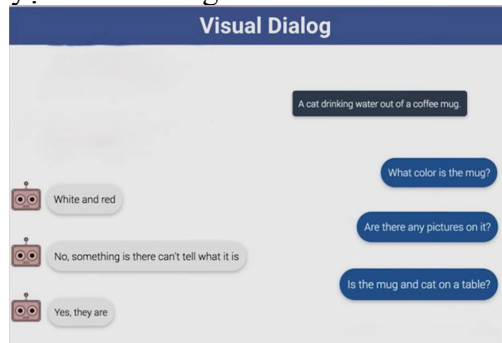
trước đó rất dễ bị vanishing

6.1. Một vài application

- Image captioning: thường sẽ kết hợp với CNN (không dùng last layer vì last layer để classify) để tạo ra matrix W trước rồi mới dùng RNN



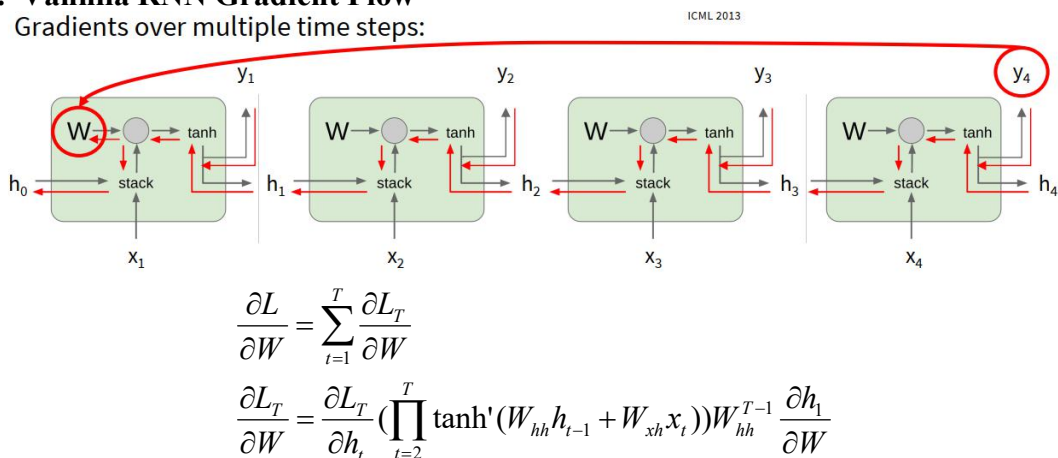
- Visual question answering (VOA)
- Visual dialog: nói chuyện về các image



- Visual Language Navigation: dùng RNN để đưa ra chuỗi các bước đi với input là cảnh thay đổi với mỗi move

7. Vannila RNN Gradient Flow

Gradients over multiple time steps:



Mà với hàm tanh đạo hàm thì hầu như sẽ bé hơn 1 tức là nếu T càng lớn thì tích các số bé hơn 1 càng nhỏ -> vanishing gradient

Còn nếu không áp dụng non-linear:

$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_T}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_t} W_{hh}^{T-1} \frac{\partial h_1}{\partial W}$$

W lớn hơn 1 thì exploding còn bé hơn 1 thì vanishing

Đối với exploding thì đơn giản ta scale gradient với norm

Còn với vanishing thì đổi kiến trúc (LSTM)

8. LSTM

Vấn đề của RNN: mọi thứ bị nhét vào 1 hidden state duy nhất -> chuỗi dài -> bản chất là vẫn lưu thông tin của các token trước đó (nhưng khó giữ đc nhiều vì sẽ bị trộn)

LSTM

Four gates

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

Cell state $\rightarrow c_t = f \odot c_{t-1} + i \odot g$

Hidden state $\rightarrow h_t = o \odot \tanh(c_t)$

Khác với RNN chỉ nhân W với hidden state trước và x_t hiện tại thì LSTM còn chia kết quả của tích này thành 4 gates:

- + i: input gate
- + f: forget gate
- + o: output gate
- + g: candidate (nội dung đề xuất ghi vào memory)

3 cổng i, f, o sẽ có dạng sigmoid -> nó sẽ quyết định bao nhiêu dữ liệu sẽ được qua
Cổng g sẽ là tanh vì nó sẽ quyết định học thêm feature (dương) này hay giảm feature này đi (âm)

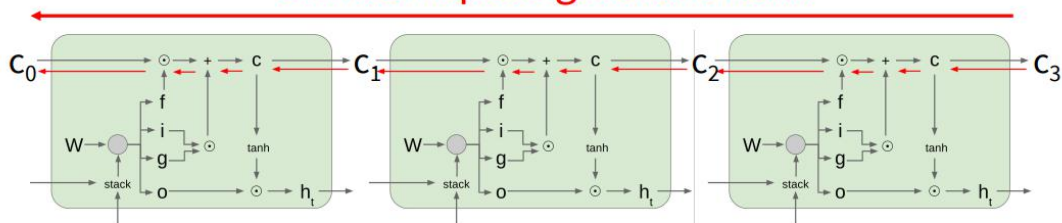
Cell state: là cái mà LSTM đang nhớ

+ f . c_{t-1} : nghĩa là nó sẽ được quên bao nhiêu từ cell trước

+ i . g: được gắn thêm vào memory bao nhiêu từ nội dung g

Hidden state sẽ là từ cổng output và $\tanh(c_t)$ -> output cuối sẽ được xây dựng từ bao nhiêu cell state hiện tại (t)

Uninterrupted gradient flow!



Backproagation sẽ chỉ phụ thuộc vào f

Nhìn kĩ cái này chưa phải là cập nhật từ loss giữa output và grouthtruth mà đang là loss giữa các Cell state

-> Tùy vào bài tùy vào loại output many to one hay many to many sẽ có các cách cập nhật khác nhau cho mỗi thành phần của W